

Stock Vision

Mark Armstrong, David Swei, Jiaxi Tian, Jia Guo

ABSTRACT

Stock prediction is an inherently challenging problem due to the volatile nature of financial markets. Traditional statistical models such as linear regression often struggle to fully capture the complex and time-dependent relationships that are present in financial data [1]. To address this limitation, this project examines the use of Long Short-Term Memory (LSTM) networks, a type of Recurrent Neural Network (RNN) designed to model sequential data like stock market data and learn long-term dependencies within it. The model used in this project is trained on the Yahoo Finance Dataset (2018 – 2023) [3], which contains historically daily stock data. This data is then processed into sequential input using a sliding window, that allows the model to learn temporal dependencies on past observations. The overall objective of this project is to evaluate the effectiveness of LSTM-based models in capturing the patterns in stock data and evaluate the accuracy of their predictions.

INTRODUCTION

Recurrent Neural Networks (RNNs) are a class of deep learning models that are designed to process sequential data. They accomplish this by maintaining a hidden state that carries information across time steps. Unlike Feed Forward Networks, RNNs form connections across time, allowing them to maintain any potential temporal dependencies. However, standard RNN models have difficulty retaining information over longer sequences due to the vanishing gradient problem, which we learned in class. By implementing a type of RNN: Long Short-Term Memory Networks (LSTMs) we can address this limitation by incorporating an additional gating mechanism that allows for the selective retaining and removing of unneeded information across time steps, making the model much more effective for longer term modeling [2].

BACKGROUND

Stock price prediction has been widely studied in both academic research and finance for a long time. Over time, a variety of approaches have been proposed to try to model and predict financial time-series data. Early methods primarily relied on statistical techniques such as moving averages and autoregressive models. While these types of models can perform decently well in stable market conditions, they assume linear relationships and tend to reflect past patterns rather than try to capture future market conditions. With advancements in deep learning, RNNs have become a popular choice and a major improvement over the statistical techniques for modeling the stock market due to its ability to model sequential data while maintaining relationships. Even still, stock price prediction remains a large problem due to events such as sudden market shifts, unexpected events, and geopolitical events. These events can completely change market behavior in unpredictable ways. This combined with the fact that models can only train on historical data, which may not necessarily reflect future market conditions keep financial time-series data highly unstable and difficult to accurately model [1].

METHODOLOGY

In this project, we implemented and compared two sequence models: a Recurrent Neural Network (RNN) and a Long Short-Term Memory (LSTM) network. The RNN was implemented from scratch in NumPy with manual backpropagation through time, while the LSTM was implemented in PyTorch with visible gating equations instead of using a black-box LSTM module. These two pipelines include problem formulation such as data organization, data preprocessing using Python built-in library, sequence construction, training the data, design the architecture of the algorithms, and lastly evaluation on how the algorithms work the best compared to others.

A. Problem Formulation

We use a lot of data from the past years to predict what the stock's next day's closing price will be. Each day represents various features such as volume, close, high, open, and low. Our goal is to use the previous 20 days of data to effectively predict the closing price for the next few days as accurately as possible. In our setup, each input has 20 consecutive days, and each day contains many features. Each input sequence contains 20 time steps, and each time step includes OHLCV features; the output will be a single value, that will be our predicted closing price for the next day.

B. Data Source and Organization

The dataset is taken from an Excel file that has a lot of daily stock data called Yahoo Finance, the one we found on the internet [3]. The Date column is converted into a specific format, and the data is sorted in chronological order to keep the time sequence. The dataset is from April 2018 to March 2023, with 1258 records of stock. Our goal is to predict the price of a close.

C. Train and Test Leakage Control

The Financial time is partitioned and trained 80% training values, and 20% testing values without shuffling to maintain temporal causality. Normalization arguments were also derived from the training dataset to prevent data leakage. RNN algorithms use Z-score normalization with sequences generated strictly within the partition boundaries. On the other hand, LSTM employs the function of min and max for splitting. The output generated was also inverse transformed using statics to ensure the exact sample of prediction.

D. Hyperparameter Summary

	RNN	LSTM
Sequence Length	20	20
Hidden Size	32	32
Learning Rate	0.01	0.01
Epochs	50	50
Loss Function	MSE	MSE
Optimizer	SGD	Adam
Train / Test	80 / 20	80 / 20
Target Column	Close*	Close*
Optimizer	SGD (per-sequence)	Adam
Learning Rate	0.001	0.01
Batch Size	1 (per-sequence)	Full batch
Training Epochs	50	100
Gradient Clipping	[-5, 5] element-wise	PyTorch autograd
Implementation	NumPy (manual BPTT)	PyTorch (explicit cell)
Random Seed	N/A	42

Figure 1 - Hyperparameter Configuration for RNN and LSTM

This figure shows the performance one-step-ahead closing price by using OHLCV features and a single chronological split. It also served as a basic benchmark; it excludes factors like trading, fees, and unpredictable changes that may happen.

RESULTS

Overall, the performance was evaluated using around 20-day multivariable period and a hidden size of 32 across a chronological 80/20 traditional split. RNN achieved better accuracy and it has better performance compared to LSTM. Contrary to the typical expectation that LSTM often outperforms a basic RNN on temporal data, the RNN achieved lower error across every metric in this experiment.

A. Quantitative Performance Metrics

	Experiment 7 Simple RNN	Experiment 11 Manual LSTM
CONFIGURATION		
Implementation	NumPy (from scratch)	Manual
Target column	Close*	Close*
Sequence length	20	20
Hidden size	32	32
Learning rate	0.003	0.01
Epochs	50	50
Loss function	MSE	MSE
Optimizer	SGD	Adam
Train / Test split	80 / 20	80 / 20
DATASET		
Dataset size	1,258	1,258
Train samples	996	990
Test samples	232	248
PERFORMANCE		
Final training loss	—	0.000687
Test MSE	159,873.90	252,649.45
Test RMSE	399.84	502.64
Test MAE	308.10	394.51
Test MAPE	—	1.2303%

Figure 2 - Test Experiments

From the Figure 2 table above, we see that the RNN outperformed the LSTM algorithm on every aspect of the error metric. This result was unexpected, because in general LSTM has more advanced architecture compared to the RNN. This shows us that implementation choices are also important, because we had LSTM with the learning rate of 0.001 which is 10 times greater than the RNN, and the closing time of 20-day input window. These two factors had a huge impact on the models' performance than the architectural differences itself.

B. Discussion

The results show that basic RNN had a lower error rate compared to the LSTM in all areas. However, this is the difference between how each model was set up, not because of the RNN being a better algorithm than LSTM [4]. Overall, each model has its own advantages and disadvantages.

ANALYSIS

The results show that the RNN model performed better than the LSTM model on this dataset. The best RNN model achieved a test MSE of 154,929.8222, RMSE of 393.6113, and MAE of 303.2032. In comparison, the best LSTM model achieved a test MSE of 252,649.4531, RMSE of 502.6425, and MAE of 394.5134. Since lower MSE, RMSE, and MAE values indicate

better prediction performance, the RNN model performed better overall in this project.

Model Comparison Table			
Model	MSE	RMSE	MAE
RNN	154929.8222	393.6113	303.2032
LSTM	252649.4531	502.6425	394.5134

Figure 3 - Test results

This result is somewhat unexpected, as LSTM models are typically designed to outperform basic RNN models in the processing of sequential data. LSTMs utilize "gating" mechanisms to control which information should be retained or discarded, thereby helping to address the problem of long-range dependencies. However, in this specific project, the input sequences span a mere 20 trading days. Given the relatively short length of these sequences, a basic RNN model may have been sufficient to capture the short-term patterns present in the stock data; consequently, the more complex architecture of the LSTM model may not have demonstrated a distinct advantage in this particular task.

Another possible reason is the size of the dataset. The dataset contains 1,258 stock records, with 990 training samples and 248 testing samples for the LSTM experiment. This is a relatively small dataset for deep learning models. Since LSTM has more internal parameters and a more complex structure than a simple RNN, it may need more data or more careful tuning to perform better. In this case, the simpler RNN may have generalized better on the test set.

The evaluation results also show that both models were able to learn the general trend of the stock data, but the RNN had smaller absolute prediction errors. The LSTM still produced a low MAPE of 1.2303%, which means its average percentage error was small.

However, compared with the RNN, its RMSE and MAE were higher, meaning its dollar-value prediction errors were larger.

Because the two models used different preprocessing and training implementations, this comparison should be interpreted as a comparison between our RNN pipeline and our

LSTM pipeline rather than a fully controlled comparison of model architecture only.

FUTURE WORK

In the future, we can improve the project by testing more settings for RNN and LSTM models. One way is that we can try more learning rates, hidden sizes, and epoch numbers to see if the LSTM model can do better.

We can also try different sequence lengths. In this project, we used the previous 20 trading days to predict the next day's Close* value. In the future, we can test different days, for example, 10 days, 30 days, or 60 days, to see which window gives the best result. Because LSTM is designed to learn long-term patterns in sequence data, using a long input may help improve its performance.

Another improvement is to use more data. Our dataset only has 1,258 records, so the LSTM model may not have enough data to show its full advantage. Using more years of stock data or testing different stocks could make the result more reliable.

We can also add more features, such as moving averages, market trends, company news, or interest rates. These factors may help the model understand stock price changes better.

Overall, future work should focus on using more data, testing more model settings, and adding more useful features to improve prediction accuracy.

CONCLUSION

In this project, we implemented the RNN and LSTM algorithms and compared two architectures, first one a vanilla RNN coded from scratch in NumPy and second LSTM network. We used these two to predict the closing price of the Yahoo Finance stock series from April 2018 – March 2023. Both models used around 20 days input window and 80/20 of the data train/test split. We have found that RNN does better

prediction in this scenario than LSTM. We can attribute this result to two factors. First, the input sequence length was relatively short, so the vanishing gradient problem may not have strongly affected the basic RNN. Second, the two models did not use fully identical hyperparameters. For example, the LSTM learning rate was ten times larger than the RNN learning rate, which may have affected the comparison.

These results show that just because a model is more advanced, it does not mean it is going to perform better. Because how you train the data and what data you use also matters just much as the algorithm. With future work we will repeat the comparison under the same hyperparameters, and use a longer window time, and add more data such as trading volume, exchange rates, and bidding amount to better assess each model's practical value for stock market predictions.

REFERENCES

- [1] C. Y. Lin and J. A. Lobo Marques, "Stock market prediction using artificial intelligence: A systematic review of systematic reviews," *Social Sciences & Humanities Open*, vol. 9, p. 100864, Jan. 2024, doi: <https://doi.org/10.1016/j.ssaho.2024.100864>.
- [2] J. Noble, "Long short-term memory," *Ibm.com*, Dec. 16, 2025. <https://www.ibm.com/think/topics/lstm>
- [3] Yahoo Finance Dataset (2018-2023)," www.kaggle.com. <https://www.kaggle.com/datasets/suruchiarora/yahoo-finance-dataset-2018-2023>
- [4] M. N. Hoda, N. Chauhan, S. Quadri, and P. R. Srivastava, Eds., *Intelligent Systems and Applications*. Singapore: Springer, 2019. [Online]. Available: https://www.google.com/books/edition/Intelligent_Systems_and_Applications/YPUWEQAAQB_AJ
- [5] A. Kara, O. K. Boyacioglu, and Ö. K. Baykan, "Predicting direction of stock price

index movement using artificial neural networks and support vector machines: The sample of the Istanbul Stock Exchange,” *Expert Systems with Applications*, vol. 38, no. 5, pp. 5311–5319, May 2011, doi: <https://doi.org/10.1016/j.eswa.2010.10.027>.